

A Cloud-Orchestrated MLOps Workflow for Reproducible Predictive Analytics

G. G.-U.¹, G. B.-C.¹, V. T.-L.¹, O. E.¹, and K. O.-G.²

¹ Institution 1, anonymized for review

² Institution 2, anonymized for review

Abstract. Moving predictive models from experimentation to production still creates practical difficulties for institutions that rely on administrative data, heterogeneous files, and manual reporting routines. This paper presents an Infrastructure-as-Code MLOps framework designed to automate the predictive model lifecycle through reproducible data ingestion, temporal model evaluation, portable deployment, and controlled natural-language interpretation of results. The proposed architecture integrates a deterministic ETL pipeline, a multi-model forecasting engine, PostgreSQL storage, Kubernetes orchestration, and Terraform-based provisioning for both local and cloud environments. The proposal was assessed using institutional publication metadata collected between 2015 and 2023. The source dataset contained 87,073 records, later aggregated into an annual time series for forecasting. Because the predictive task was based on a short annual sequence, the results are treated as a bounded validation scenario rather than a broad claim of model superiority. The experimental workflow compared Poisson GLM, Ridge, Lasso, Gradient Boosting, Random Forest, and ETS models using expanding-window validation and a composite selection criterion based on point error, temporal stability, and interval calibration. ETS obtained the best composite score, while Poisson GLM reached lower point-error values in the final horizon. The ETL process normalized 16,599 journal categories in 27.91 seconds, showing adequate behavior for institutional datasets of similar size. The reporting layer, implemented with GPT-4o-mini, converted structured outputs into constrained narrative summaries. The study offers an auditable MLOps workflow for institutional predictive analytics and identifies limits associated with short temporal series.

Keywords: MLOps · Infrastructure as Code · predictive analytics · Kubernetes · Terraform · reproducibility · cloud computing · large language models

1 Introduction

Machine learning models often fail at the boundary between experimentation and operation. A model may perform well during development, yet behave inconsistently when exposed to heterogeneous files, changing schemas, manual deployment steps, or execution environments that differ from testing conditions [1,

2, 4, 6, 8]. This problem is frequent in institutional analytics, where data usually come from administrative repositories, spreadsheets, and semi-structured records created for reporting, not for predictive modeling.

MLOps addresses this gap by connecting data preparation, model evaluation, deployment, monitoring, and maintenance through repeatable workflows [5, 6]. In production settings, predictive accuracy alone is not enough. The workflow must preserve traceability, control preprocessing decisions, reduce configuration drift, and allow the same analytical services to be reproduced across environments [7, 8]. Automated ingestion is especially important when files contain inconsistent headers, empty fields, mixed data types, or temporal values outside the study range [9, 10].

This paper presents an Infrastructure-as-Code MLOps framework for institutional predictive analytics, developed under a Design Science Research approach. The framework integrates deterministic data ingestion, PostgreSQL persistence, multi-model forecasting, Streamlit visualization, Kubernetes orchestration, Terraform provisioning, and a constrained LLM reporting layer. Kubernetes coordinates the analytical services, Terraform recreates the infrastructure in local and cloud environments, and the LLM component converts structured statistical outputs into short reports without altering the computed results [11–21].

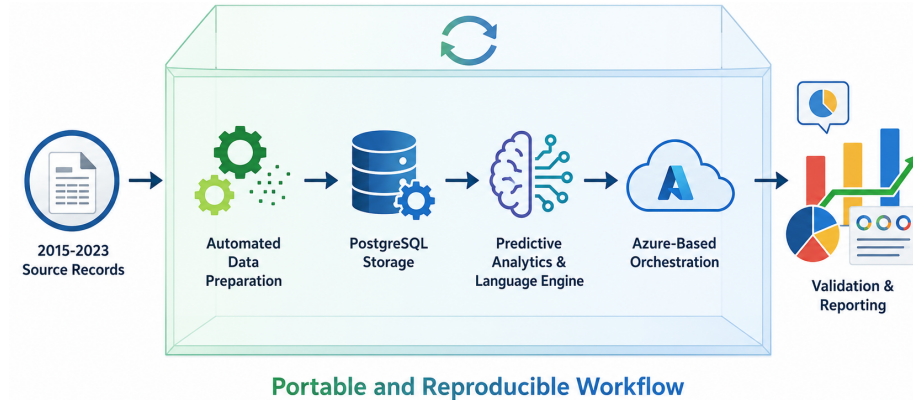


Fig. 1. Portable MLOps workflow for data preparation, prediction, deployment, and reporting.

The contribution of this work is a reproducible workflow that separates ingestion, storage, model evaluation, deployment, and reporting into auditable components. The predictive stage uses expanding-window validation and a composite score that combines point error, temporal stability, and interval calibration. The evaluation uses publication metadata from 2015 to 2023; since the forecasting task is based on annual aggregates, the results are interpreted as operational validation of the framework, not as a universal ranking of forecasting models.

2 Materials and Methods

This study follows a Design Science Research (DSR) approach [26]. The artifact was built to support institutional predictive analytics through four stages: architecture design, deterministic data ingestion, temporal model evaluation, and reproducible deployment through Infrastructure as Code (IaC).

2.1 Architecture Design and Orchestration

The proposed architecture organizes the predictive workflow as containerized services. Figure 2 shows how source records from 2015 to 2023 are processed by the HHDS ingestion module, stored in PostgreSQL 16, analyzed by the forecasting engine, displayed in Streamlit, and summarized by the controlled LLM reporting layer.

Kubernetes coordinates the services, while StatefulSets support the database workload. The local version was deployed with Docker Compose, and the cloud version was provisioned on Microsoft Azure through Terraform. This design keeps the same service structure across both environments.

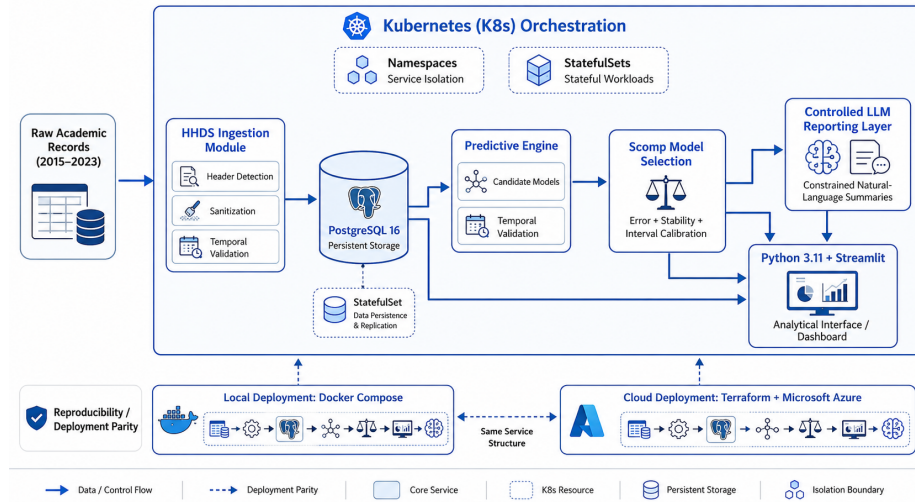


Fig. 2. MLOps workflow for ingestion, storage, forecasting, deployment, and reporting.

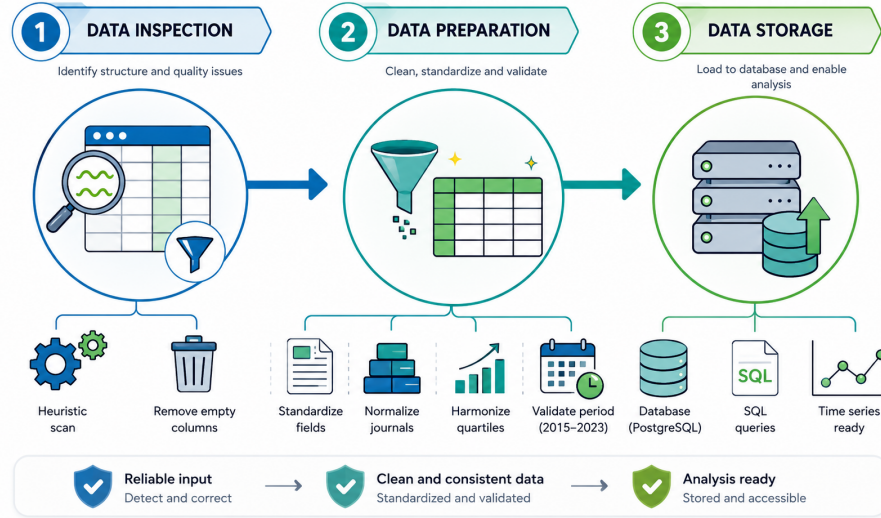
2.2 Deterministic Ingestion Pipeline and Normalization

The ingestion pipeline was designed to handle heterogeneous academic records with minimal manual editing. The dataset contains publication metadata from 2015 to 2023 and was obtained from the official repository *Base de Datos de Artículos Publicados UEP En Revistas Indexadas — Conjunto de Datos — Datos Abiertos Ecuador* [1]. The expected fields were aligned with the *Diccionario SIIES*, used as the reference schema.

Table 1. SIIES variables used by the ingestion pipeline.

Header	Description
AÑO	Publication year.
TIPO	Publication type.
NOMBRE UNIVERSIDAD	University or polytechnic school.
TIPO FINANCIAMIENTO	Institutional funding type.
PROVINCIA UNIVERSIDAD	University province.
BASE DATOS INDEXADA	Indexing database.
NOMBRE REVISTA	Journal name.
NOMBRE ARTICULO	Article title.
CAMPO AMPLIO	Broad knowledge field.
CAMPO ESPECIFICO	Specific knowledge field.
CAMPO DETALLADO	Detailed knowledge field.

HHDS scans the first $N = 20$ rows of each spreadsheet, detects the most likely header, removes unusable columns, validates the year field, and loads the sanitized data into PostgreSQL 16.

**Fig. 3.** Deterministic ingestion pipeline. Raw spreadsheets are inspected, prepared, and stored as a PostgreSQL 16 dataset ready for queries and time-series analysis.

Let D be a raw spreadsheet, r_i the i -th row, and C^* the expected schema. The header score is:

$$\rho_i = \frac{1}{|C^*|} \sum_{c \in C^*} \mathbb{I}[c \in \text{norm}(r_i)]. \quad (1)$$

The selected header is:

$$h^* = \arg \max_{1 \leq i \leq N} \rho_i. \quad (2)$$

The sanitized dataset keeps valid fields and records within the study period:

$$D_s = \{x_{tj} : 2015 \leq year_t \leq 2023 \wedge name(x_j) \in C^* \wedge Var(x_j) > 0\}. \quad (3)$$

Algorithm 1 Heuristic Header Detection and Sanitization (HHDS)

Require: Raw spreadsheet D ; expected schema C^* ; scan depth $N = 20$; valid temporal range $[2015, 2023]$

Ensure: Sanitized dataset D_s

- 1: Initialize $HeaderScores \leftarrow \emptyset$
 - 2: **for** $i = 1$ **to** N **do**
 - 3: $r_i \leftarrow$ extract row i from D
 - 4: $\rho_i \leftarrow \frac{1}{|C^*|} \sum_{c \in C^*} \mathbb{I}[c \in \text{norm}(r_i)]$
 - 5: $HeaderScores.add(i, \rho_i)$
 - 6: **end for**
 - 7: $h^* \leftarrow \arg \max_i \rho_i$
 - 8: Assign row h^* as official header
 - 9: Remove rows located above h^*
 - 10: **for** each column x_j in D **do**
 - 11: **if** $name(x_j) \notin C^*$ or $Var(x_j) = 0$ or x_j is empty **then**
 - 12: Drop column x_j
 - 13: **end if**
 - 14: **end for**
 - 15: **for** each row t in D **do**
 - 16: **if** $year_t < 2015$ or $year_t > 2023$ or $year_t$ is non-numeric **then**
 - 17: Drop row t
 - 18: **end if**
 - 19: **end for**
 - 20: Normalize categorical fields and enforce temporal data type
 - 21: Load sanitized dataset into PostgreSQL 16
 - 22: **return** D_s
-

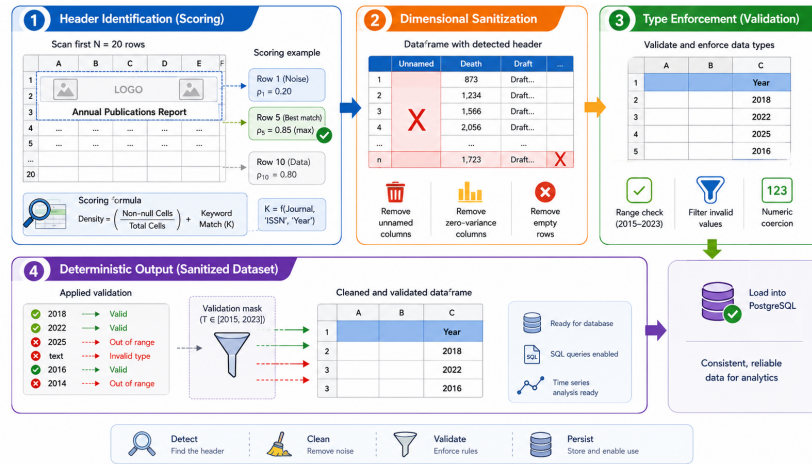


Fig. 4. HHDS workflow. The algorithm detects the header, removes unusable fields, validates year values, and stores the sanitized output in PostgreSQL 16.

2.3 Experimental Predictive Modeling Framework

The predictive task was treated as an annual univariate forecasting problem. Although the source table contains 87,073 records, the effective forecasting series contains nine annual observations from 2015 to 2023.

Let y_t be the number of publications in year t :

$$y_t = \sum_{i=1}^n \mathbb{I}(\text{year}_i = t). \quad (4)$$

The candidate models were:

$$M = \{\text{Poisson GLM, Linear, Ridge, Lasso, GBR, RF, ETS}\}. \quad (5)$$

Evaluation used expanding-window temporal validation with $K = 3$ folds. Models were compared through MAE, RMSE, MAPE, conformal intervals, and the composite score S_{comp} :

$$S_{\text{comp},m} = \overline{MAPE}_m + \sigma(MAPE_m) + \Delta Cov_m. \quad (6)$$

Lower values indicate a better balance between point error, temporal stability, and interval calibration.

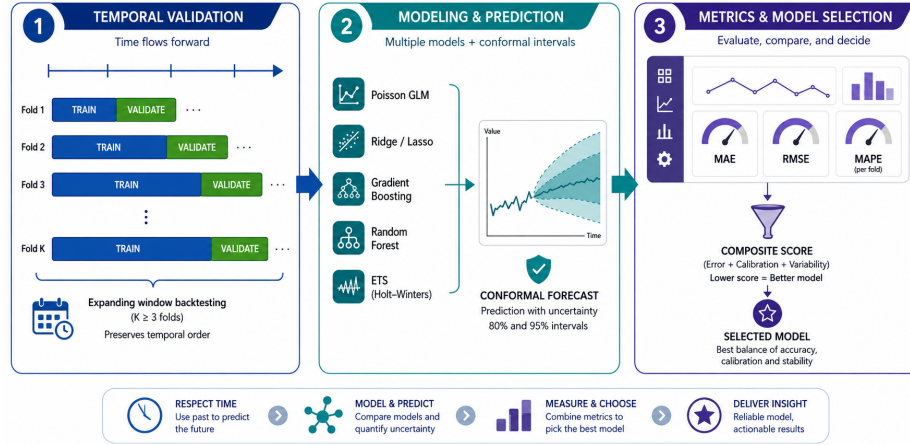


Fig. 5. Predictive modeling workflow. The annual series is evaluated through temporal splits, candidate models, conformal intervals, error metrics, and the composite score used to select the final model.

Algorithm 2 Temporal Model Evaluation and Composite Selection

Require: Annual time series $Y = \{y_{2015}, \dots, y_{2023}\}$; model set M ; folds $K = 3$; significance levels $A = \{0.20, 0.05\}$

Ensure: Selected model B ; final forecast $\hat{Y}_{2026}$; conformal intervals CI

- 1: Initialize *ResultsRegistry* $\leftarrow \emptyset$
- 2: **for** each model $m \in M$ **do**
- 3: Initialize *FoldErrors* $\leftarrow []$
- 4: Initialize *ResidualsOOF* $\leftarrow []$
- 5: **for** $k = 1$ **to** K **do**
- 6: Build $D_{\text{train}}^{(k)}$ and $D_{\text{valid}}^{(k)}$ using chronological order
- 7: $\hat{m}^{(k)} \leftarrow \text{FIT}(m, D_{\text{train}}^{(k)})$
- 8: $\hat{y}^{(k)} \leftarrow \text{PREDICT}(\hat{m}^{(k)}, D_{\text{valid}}^{(k)})$
- 9: $\text{MAPE}_m^{(k)} \leftarrow \text{MAPE}(y^{(k)}, \hat{y}^{(k)})$
- 10: *FoldErrors*.append($\text{MAPE}_m^{(k)}$)
- 11: *ResidualsOOF*.extend($|y^{(k)} - \hat{y}^{(k)}|$)
- 12: **end for**
- 13: $\overline{\text{MAPE}}_m \leftarrow \text{MEAN}(\text{FoldErrors})$
- 14: $\sigma(\text{MAPE}_m) \leftarrow \text{STDDDEV}(\text{FoldErrors})$
- 15: Initialize $\Delta\text{Cov}_m \leftarrow 0$
- 16: **for** each $\alpha \in A$ **do**
- 17: $q_{m,\alpha} \leftarrow Q_{1-\alpha}(\text{ResidualsOOF})$
- 18: $\text{Cov}_{m,\alpha} \leftarrow \text{EMPIRICALCOVERAGE}(q_{m,\alpha})$
- 19: $\Delta\text{Cov}_m \leftarrow \Delta\text{Cov}_m + |\text{Cov}_{m,\alpha} - (1 - \alpha)|$
- 20: **end for**
- 21: $S_{\text{comp},m} \leftarrow \overline{\text{MAPE}}_m + \sigma(\text{MAPE}_m) + \Delta\text{Cov}_m$
- 22: *ResultsRegistry*.add($m, S_{\text{comp},m}, \text{ResidualsOOF}$)
- 23: **end for**
- 24: $B \leftarrow \arg \min_m S_{\text{comp},m}$
- 25: $B_{\text{final}} \leftarrow \text{FIT}(B, Y)$
- 26: $\hat{Y}_{2026} \leftarrow \text{PREDICT}(B_{\text{final}}, 2026)$
- 27: $CI \leftarrow \text{CONFORMALINTERVALS}(\hat{Y}_{2026}, \text{ResultsRegistry}[B].\text{ResidualsOOF})$
- 28: **return** $B_{\text{final}}, \hat{Y}_{2026}, CI$

2.4 Implementation of the Interpretability Layer

The reporting layer receives a JSON payload with historical values, forecasts, intervals, error metrics, and the selected model. GPT-4o-mini was accessed through a Python wrapper with temperature set to 0.3. The prompt restricted the output to the provided values and generated short summaries for institutional users.

2.5 Validation of Reproducibility via Infrastructure as Code

Reproducibility was assessed through two deployments: a local version with Docker Compose and a cloud version provisioned with Terraform on Microsoft Azure. The cloud deployment used Azure Container Apps and PostgreSQL Flexible Server. The assessment checked whether the same ingestion module, database schema, forecasting engine, dashboard, and reporting layer could run with the same service structure in both environments.

3 Results

3.1 Pipeline Execution and Deployment Reproducibility

The HHDS pipeline processed 87,073 records and identified 16,599 unique journal categories. Normalization took 27.91 seconds, close to 3,120 records per second. This value reports the execution time for the evaluated dataset, not a general scalability benchmark.

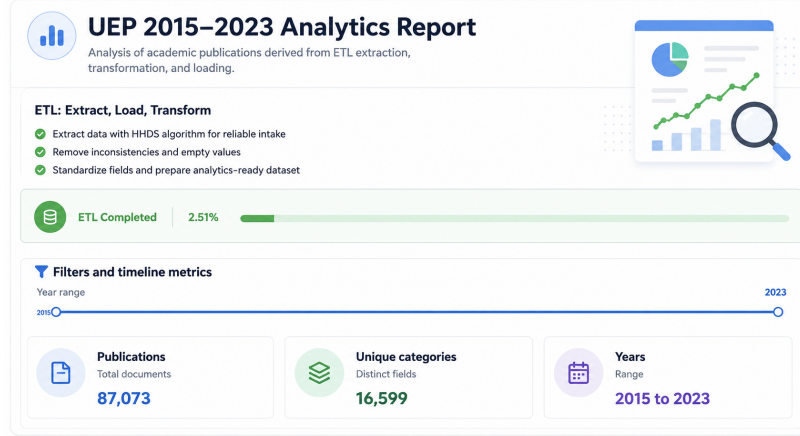


Fig. 6. ETL dashboard. The pipeline processed 87,073 records, identified 16,599 categories, covered 2015–2023, and completed normalization in 27.91 seconds.

The cloud deployment was recreated through Terraform. Figure 7 shows the provisioned Azure resources: dashboard application, inference service, PostgreSQL server, and container registry. The same service layout was kept in Docker Compose and Azure.

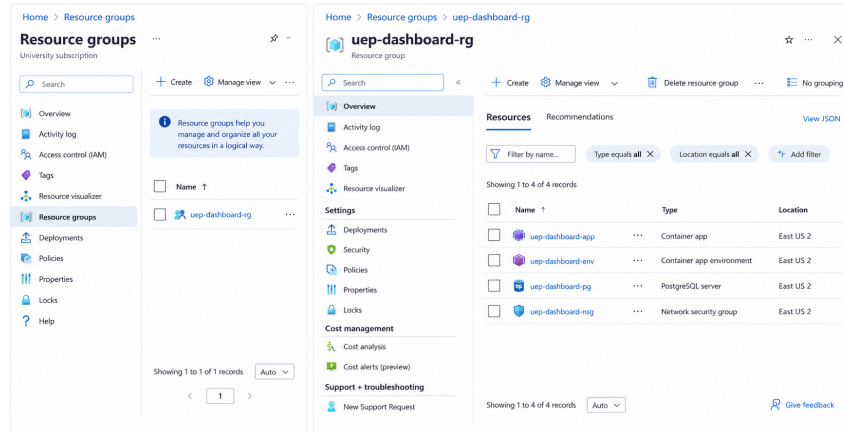


Fig. 7. Azure deployment. Terraform provisioned the dashboard, inference service, PostgreSQL server, and container registry with parity to the local Docker Compose setup.

3.2 Descriptive Analysis of Publication Records

The annual record count increased from 3,936 in 2015 to 15,229 in 2023. This pattern describes the administrative dataset and should not be treated as direct evidence of changes in scientific productivity.

The journal distribution was concentrated in a small group of venues. Figure 8 shows that the top categories account for a high share of the records.

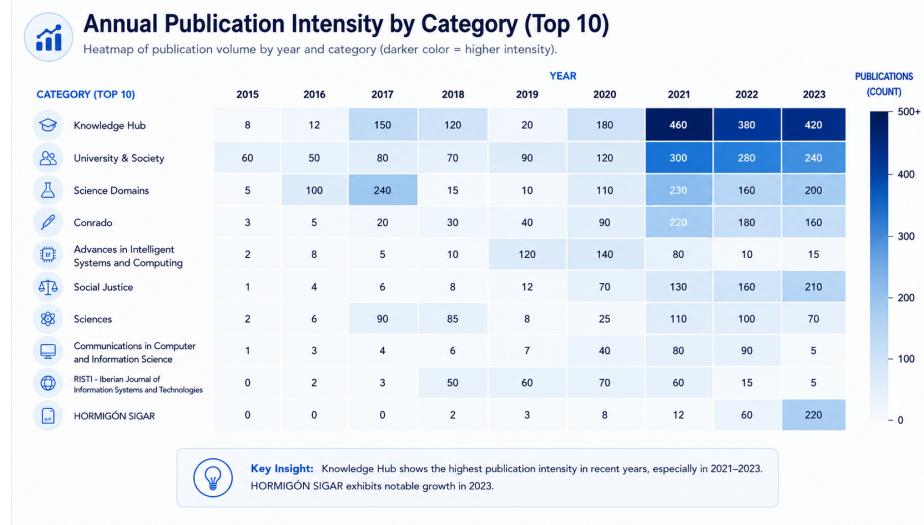


Fig. 8. Publication intensity by category. The heatmap shows yearly record concentration across the top ten journal categories from 2015 to 2023.

3.3 Forecasting Sample and Model Outputs

The forecasting task used annual counts after aggregation. Thus, the model comparison was based on nine observations, not on the 87,073 administrative rows.

The Poisson GLM projected 23,373.87 records for 2026, with an 80% interval from 23,177.93 to 23,569.80.

Table 2. Poisson GLM projections with 80% conformal intervals.

Year	Forecast	LPI (80%)	UPI (80%)
2024	17,860.00	17,688.72	18,031.27
2025	20,431.77	20,248.58	20,614.96
2026	23,373.87	23,177.93	23,569.80



Fig. 9. Publication forecast. Observed annual values from 2015 to 2023 are shown with Poisson GLM projections for 2024–2026 and 80% conformal intervals.

3.4 Model Comparison and Composite Selection

Table 3 reports the 2026 projections and validation errors. The error columns summarize validation behavior from the historical series; they are not observed errors for 2026.

Table 3. Model projections for 2026 and validation error estimates.

Model	Forecast	LPI	UPI	MAE	RMSE	MAPE
Poisson	23,373.87	23,177.93	23,569.80	527.66	620.25	3.5
Linear	18,256.56	17,280.93	19,232.20	4,137.43	4,255.09	28.3
Ridge	19,321.13	18,321.15	20,321.11	2,552.31	2,605.01	17.5
Lasso	18,256.57	17,280.94	19,232.21	4,137.42	4,255.08	28.3
GBR	15,228.85	15,228.73	15,228.97	3,415.09	3,491.41	23.4
RF	14,402.56	13,764.57	15,040.55	3,737.28	3,807.14	25.6
ETS	18,567.71	17,590.52	19,544.90	1,626.95	1,632.13	11.2

Note: LPI and UPI correspond to 80% conformal intervals.

Poisson GLM obtained the lowest point-error values. ETS, however, obtained the best composite score because S_{comp} also considers temporal stability and interval calibration.

Table 4. Model ranking under S_{comp} .

Model	Score	Model	Score	Model	Score
ETS	4.33	RF	11.75	GBR	12.59
Ridge	20.95	Lasso	31.46	Linear	31.49
Poisson	34.19				

This difference explains the final selection: Poisson GLM was stronger in point error, while ETS offered the best balance under the proposed governance rule.

3.5 Interpretability Layer Output

The reporting layer converted the selected model output into short explanations. Figure 10 presents the ETS forecast, conformal intervals, LLM annotations, and the generated recommendation. The text was produced from the structured JSON payload used by the reporting service.

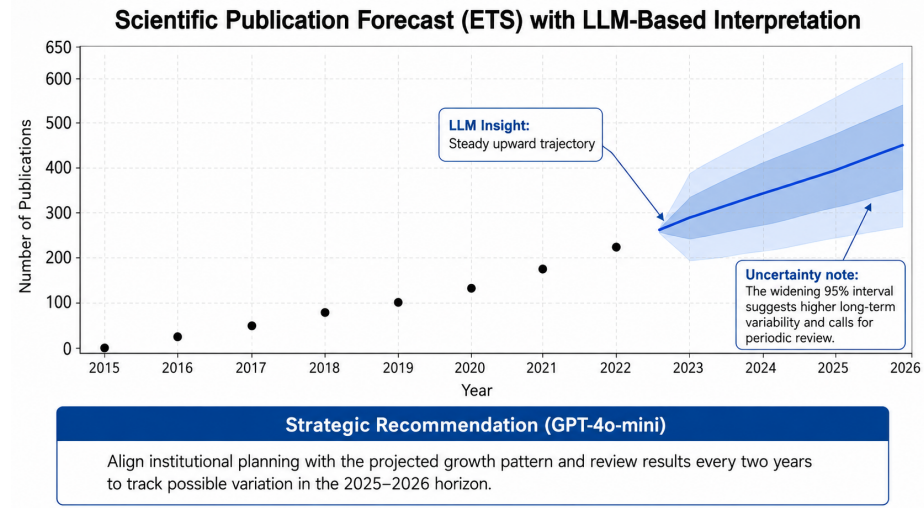


Fig. 10. LLM reporting layer. The ETS forecast is shown with conformal intervals and short annotations generated from the structured statistical payload.

4 Discussion

The results show that the proposed architecture can organize the predictive model lifecycle from ingestion to deployment and reporting. Its contribution is the workflow connection among HHDS, PostgreSQL, temporal validation, Kubernetes, Terraform, Streamlit, and the controlled LLM layer. The forecasting results require caution. The source table contains 87,073 records, but the annual series contains only nine observations. For this reason, the experiment validates the workflow behavior rather than proving that one model will generalize across longer or different datasets.

4.1 Model Governance and Forecast Interpretation

The comparison between Poisson GLM and ETS shows why model selection should not rely only on point error. Poisson GLM had the lowest MAE, RMSE,

and MAPE, while ETS obtained the best S_{comp} . In a short series, each validation point can strongly affect the ranking, so stability and interval calibration were included in the selection rule.

ETS was selected because it performed best under the proposed criterion. Poisson GLM remains a strong candidate and should be tested again with simple baselines such as naive, drift, log-linear trend, and simple exponential smoothing models.

4.2 Infrastructure Reproducibility and Pipeline Governance

Docker Compose and Terraform allowed the same service structure to be deployed locally and in Microsoft Azure. HHDS also improved auditability by detecting headers, filtering fields, validating years, and loading the output into PostgreSQL. The ETL time of 27.91 seconds confirms that the pipeline worked for this dataset. Broader infrastructure claims would require repeated runs, hardware reports, memory measurements, load tests, latency tracking, and failure recovery tests.

4.3 Controlled LLM Reporting Layer

The LLM component was used only for reporting. It did not create forecasts, select models, update the database, or alter statistical results. Its output was restricted to the values included in the JSON payload. A stronger evaluation of this layer should include expert review, comparison with rule-based templates, stability tests, and hallucination-rate measurement.

4.4 Limitations

The main limitation is the short annual series. The study also uses one dataset, so the observed growth may reflect publication behavior, indexing practices, institutional incentives, reporting rules, or data availability. The conformal intervals were estimated from few validation residuals, and the infrastructure evaluation reports service parity rather than stress or fault-tolerance performance. The LLM layer also requires a formal evaluation before being treated as a validated interpretability method.

5 Conclusions

This study presented a cloud-based MLOps architecture for institutional predictive analytics. The workflow connects deterministic ingestion, temporal validation, persistent storage, reproducible deployment, and constrained narrative reporting.

HHDS processed 87,073 records and normalized 16,599 journal categories in 27.91 seconds. The forecasting experiment showed that the administrative record

count and the effective annual series must be interpreted separately: the source table is large, but the forecasting sample contains nine observations.

Poisson GLM obtained the best point-error values, while ETS achieved the lowest S_{comp} . Therefore, ETS was selected under the proposed governance rule, not as a universally superior model.

The same service structure was deployed locally with Docker Compose and in Microsoft Azure through Terraform. This supports configuration-level portability, although stronger claims require more infrastructure testing.

The GPT-4o-mini layer generated short summaries from structured statistical outputs. Its role was limited to reporting and decision-support communication. Future work should add simple forecasting baselines, evaluate the workflow on broader datasets, test the infrastructure under load, and compare LLM reporting against deterministic templates.

References

1. SENESCYT. Base de datos de artículos publicados UEP en revistas indexadas — Conjunto de datos — Datos Abiertos Ecuador, 2024. Available online: <https://datosabiertos.gob.ec/dataset/base-de-datos-de-articulos-publicados-uep-en-revistas-indexadas> (accessed on 10 January 2026).
2. Kreuzberger, D.; Kühl, N.; Hirschl, S. Machine Learning Operations (MLOps): Overview, definition, and architecture. *IEEE Access* **2023**, *11*, 31866–31879. <https://doi.org/10.1109/ACCESS.2023.3262138>
3. Zhang, Y.; Shen, C.; Shang, X.; Huang, L.; Fan, C. Towards trustworthy and aligned machine learning: A data-centric survey with causality perspectives. *IEEE Trans. Knowl. Data Eng.* **2024**, *36*, 4512–4531. <https://doi.org/10.48550/arXiv.2307.16851>
4. Diaz-De-Arcaya, J.; Torre-Bastida, A.I.; Zárate, G.; Miñón, R.; Almeida, A. A joint study of the challenges, opportunities, and roadmap of MLOps and AIOps: A systematic survey. *ACM Comput. Surv.* **2024**, *56*, 1–36. <https://doi.org/10.1145/3617833>
5. Boukaf, M.; Fadli, F.; Meskin, N. A Comprehensive Review of Digital Twin Technology in Building Energy Consumption Forecasting. *IEEE Access* **2024**. <https://doi.org/10.1109/ACCESS.2024.3498107>
6. Eken, B.; Pallewatta, S.; Tran, N.K.; Tosun, A.; Babar, M.A. A multivocal review of MLOps practices, challenges and open issues. *ACM Comput. Surv.* **2025**, *58*, 1–38. <https://doi.org/10.1145/3747346>
7. Foidl, H.; Golendukhina, V.; Ramler, R.; Felderer, M. Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers. *Journal of Systems and Software* **2024**, *207*, 111855. <https://doi.org/10.1016/j.jss.2023.111855>
8. Bayram, F.; Ahmed, B.S. Towards trustworthy machine learning in production: An overview of the robustness in MLOps approach. *ACM Comput. Surv.* **2025**, *57*, 111. <https://doi.org/10.1145/3708497>
9. Toka, L.; Dobreff, G.; Fodor, B.; Sonkoly, B. Machine Learning-Based Scaling Management for Kubernetes Edge Clusters. *IEEE Trans. Netw. Serv. Manag.* **2021**, *18*, 958–972. <https://doi.org/10.1109/TNSM.2021.3052837>

10. Woźniak, A.P.; Milczarek, M.; Woźniak, J. MLOps Components, Tools, Process, and Metrics: A Systematic Literature Review. *IEEE Access* **2025**, *13*, 22166–22175. <https://doi.org/10.1109/ACCESS.2025.3534990>
11. Zarour, M.; Alzabut, H.; Al-Sarayreh, K.T. MLOps best practices, challenges and maturity models: A systematic literature review. *Information and Software Technology* **2025**, *183*, 107733. <https://doi.org/10.1016/j.infsof.2025.107733>
12. Ahmed, B.S.; Azzalin, T.; Kassler, A.; Thore, A.; Lindbäck, H. Smart manufacturing: MLOps-enabled event-driven architecture for enhanced control in steel production. *Journal of Systems and Software* **2025**, *230*, 112542. <https://doi.org/10.1016/j.jss.2025.112542>
13. Do, T.V.; Do, N.H.; Rotter, C.; Lakshman, T.V.; Biró, C.; Bérczes, T. Properties of Horizontal Pod Autoscaling Algorithms and Application for Scaling Cloud-Native Network Functions. *IEEE Trans. Netw. Serv. Manag.* **2025**. <https://doi.org/10.1109/TNSM.2025.3532121>
14. Zhang, Y.; Chen, W.; Zhu, Z.; Qin, D.; Sun, L.; Wang, X.; Jin, R. Addressing concept shift in online time series forecasting: Detect-then-adapt. In *Advances in Neural Information Processing Systems*; Curran Associates: Red Hook, NY, USA, 2024. <https://doi.org/10.48550/arXiv.2403.14949>
15. Wang, X.; Tang, Z.; Guo, J.; Meng, T.; Wang, C.; Wang, T.; Jia, W. Empowering Edge Intelligence: A Comprehensive Survey on On-Device AI Models. *ACM Computing Surveys* **2025**, *57*(9). <https://doi.org/10.1145/3724420>
16. Safdar, M.; Paul, P.P.; Lamouche, G.; Wood, G.; Zimmermann, M.; Hannesen, F.; Bescond, C.; Wanjara, P.; Zhao, Y.F. Fundamental requirements of a machine learning operations platform for industrial metal additive manufacturing. *Computers in Industry* **2024**, *154*, 104037. <https://doi.org/10.1016/j.compind.2023.104037>
17. Ramesh, G.; Vaikunta Pai, T.; Birau, R.; Poojary, K.K.; Abhay; Shingad, A.R.; Sowjanya, N.; Popescu, V.; Mitroi, A.T.; Nioata, R.M.; Kiran Raj, K.M. A comprehensive review on scaling machine learning workflows using cloud technologies and DevOps. *IEEE Access* **2025**, *13*, 148559–148594. <https://doi.org/10.1109/ACCESS.2025.3563201>
18. Brito Casanova, G.J. Servicio web basado en analítica de datos en la nube para instituciones de educación superior. M.S. Thesis, Quevedo State Technical University, Quevedo, Ecuador, 2025.
19. Chatterjee, A.; Ahmed, B.S.; Hallin, E.; Engman, A. Testing of machine learning models with limited samples: An industrial vacuum pumping application. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, Singapore, 14–18 November 2022; pp. 1280–1290. <https://doi.org/10.1145/3540250.3560879>
20. de Almeida, J.G.; Messiou, C.; Withey, S.J.; Matos, C.; Koh, D.-M.; Papanikolaou, N. Medical machine learning operations: a framework to facilitate clinical AI development and deployment in radiology. *European Radiology* **2025**, *35*, 6828–6841. <https://doi.org/10.1007/s00330-025-11654-6>
21. Testi, M.; Ballabio, M.; Frontoni, E.; Iannello, G.; Moccia, S.; Soda, P.; Vessio, G. MLOps: A taxonomy and a methodology. *IEEE Access* **2022**, *10*, 63606–63618. <https://doi.org/10.1109/ACCESS.2022.3181730>
22. Najafabadi, F.A.; Bogner, J.; Gerostathopoulos, I.; Lago, P. An architectural perspective on MLOps: Structures, processes, tools, and stakeholders. *Information and Software Technology* **2026**, *193*, 108029. <https://doi.org/10.1016/j.infsof.2026.108029>

23. Berberi, L.; Kozlov, V.; Nguyen, G.; Sáinz-Pardo Díaz, J.; Calatrava, A.; Moltó, G.; Tran, V.; López García, Á. Machine learning operations landscape: Platforms and tools. *Artif. Intell. Rev.* **2025**, *58*, 148. <https://doi.org/10.1007/s10462-025-11121-4>
24. Butt, T.; Iqbal, M.; Arshad, N. From policy to pipeline: A governance framework for AI development and operations pipelines. *IEEE Access* **2025**, *13*, 52310–52325. <https://doi.org/10.1109/ACCESS.2025.3549832>
25. John, M.M.; Holmström Olsson, H.; Bosch, J. An empirical guide to MLOps adoption: Framework, maturity model and taxonomy. *Information and Software Technology* **2025**, *183*, 107725. <https://doi.org/10.1016/j.infsof.2025.107725>
26. Idowu, S.; Osman, O.; Strüder, D.; Berger, T. Machine learning experiment management tools: A mixed-methods empirical study. *Empir. Softw. Eng.* **2024**, *29*, 108. <https://doi.org/10.1007/s10664-024-10481-5>
27. Zhang, Y.; Wu, Y.; Wang, T.; Ding, B.; Wang, H. What problems are MLOps practitioners talking about? A study of discussions in Stack Overflow forum and GitHub projects. *Information and Software Technology* **2025**, *185*, 107768. <https://doi.org/10.1016/j.infsof.2025.107768>
28. Xu, C.; Xie, Y. Sequential predictive conformal inference for time series. In *Proceedings of the 40th International Conference on Machine Learning*; PMLR: Honolulu, HI, USA, 2023; Volume 202, pp. 38707–38727. <https://proceedings.mlr.press/v202/xu23r.html>
29. Angelopoulos, A.N.; Bates, S. Conformal prediction: A gentle introduction. *Found. Trends Mach. Learn.* **2023**, *16*, 494–591. <https://doi.org/10.1561/22000000101>
30. Oliveira, R.I.; Orenstein, P.; Ramos, T.; Romano, J.V. Split conformal prediction and non-exchangeable data. *J. Mach. Learn. Res.* **2024**, *25*, 225:1–225:38. <https://jmlr.org/papers/v25/23-1553.html>
31. Sun, S.; Yu, R. Conformal prediction for time-series forecasting with change points. In *Advances in Neural Information Processing Systems*; Curran Associates: Red Hook, NY, USA, 2025; Volume 38. <https://openreview.net/forum?id=HgLaVgCpCl>
32. Han, E.; Huang, C.; Wang, K. Model assessment and selection under temporal distribution shift. In *Proceedings of the 41st International Conference on Machine Learning*; PMLR: Honolulu, HI, USA, 2024; Volume 235, pp. 17374–17392. <https://proceedings.mlr.press/v235/han24b.html>